

Equipment Cleaning Log

A traceable cleaning log for pharmaceutical manufacturing equipment, with field-level audit trail

Field	Detail
Prepared by	Aravind R · aravind.r194@gmail.com
Prepared for	Leucine — TNN Technologies Private Limited, Bangalore
Purpose	Take-home assignment, Full-Stack Engineer
Document version	1.0
Date	08 September 2026
Scope	All core requirements plus all four stretch goals
Companion artefacts	Git repository, README.md, NOTES.md

This document states the data model, the audit mechanism, and the complete requirement set for the assignment. It is the design record that accompanies the source repository; the README covers installation and the NOTES file records decisions taken during implementation.

Contents

1. Purpose and Scope
 2. Technology Stack and Key Decisions
 3. Domain Entities
 4. Entity-Relationship Diagram
 5. Enumerations
 6. Column-Level Schema
 7. The Audited Write Path
 8. API Surface
 9. Functional Requirements
 10. Non-Functional Requirements
 11. Test Coverage
 12. Assumptions
 13. Exclusions
 14. Deliverables
-

Sections 3 to 7 describe the data model and the audit mechanism. Sections 9 to 11 carry the requirement set, with each functional requirement traced to its origin in the assignment brief.

1. Purpose and Scope

Equipment used in pharmaceutical manufacturing must be cleaned between production runs, and every record of that cleaning must be traceable for regulatory audit. This application implements a small, complete slice of that problem: a log of cleaning events against a register of equipment, where every change to a cleaning record is preserved as a field-level history of who changed what, when, and from which value to which.

The functional boundary is deliberately narrow. The system records and verifies cleaning events; it does not schedule them, validate cleaning efficacy, or hold electronic signatures. Section 14 states what was excluded and why.

1.1 Scope taken

- **Core requirements** — the data model, the API with pagination and filtering, the audit trail with a retrieval endpoint, the React front-end, and a focused test suite. All are in scope and specified below.
- **Stretch goals** — all four were taken: authentication with a real current user, server-side validation with structured error responses, Docker Compose for single-command startup, and keyset (cursor) pagination in place of offset.

Taking authentication first was a deliberate ordering choice: it makes the audit "who" a genuine foreign key rather than a client-supplied string, which materially changes the integrity of the audit trail and therefore the shape of the data model.

2. Technology Stack and Key Decisions

Layer	Choice and rationale
Database	PostgreSQL 16. Native enum types, transactional DDL, and row-level locking are all used directly by the design.
Data access	Raw parameterized SQL via the pg driver — no ORM. Chosen for full control over the transaction and locking semantics the audit trail depends on, for explicit SQL that can be defended line by line, and because keyset pagination reads naturally as SQL rather than as an ORM escape hatch. The cost, accepted knowingly, is hand-written row-to-type mapping at the repository boundary.
API	Node.js with TypeScript and Express. Layered as route → schema → service → repository; no SQL in route handlers and no HTTP types in services.
Validation	Zod schemas at every boundary, producing a single error envelope with per-field detail.
Front-end	React with TypeScript, built with Vite. No component library and no global state manager — a query cache and local component state are sufficient at this size.
Testing	Vitest for unit tests; Supertest against a real PostgreSQL instance for API tests. The database is never mocked for pagination or audit behaviour, since those are precisely the behaviours a mock would fabricate.
Runtime	Docker Compose brings up PostgreSQL, the API, and the web client; migrations and the seed script run on startup.

3. Domain Entities

The assignment names two entities and describes a third. Two design decisions expand the model to five tables.

Entity	Table	Responsibility
User	<code>users</code>	An operator, supervisor, or administrator. Introduced by the authentication stretch goal so that the person who performed a cleaning, the person who verified it, and the person who made any change are all real referenced identities rather than free text.
Equipment	<code>equipment</code>	The asset being cleaned, identified on the plant floor by a unique asset code. Carries an active/retired lifecycle instead of being deletable.
Cleaning Record	<code>cleaning_records</code>	One cleaning event performed on one piece of equipment. The only entity that is edited after creation, and therefore the only entity that is audited.
Audit Event	<code>audit_events</code>	One act of change. Records who made it, when, what kind of change it was, and which row it targeted. One API mutation produces exactly one audit event.
Audit Field Change	<code>audit_field_changes</code>	One row per field that actually moved during that act, holding the old value and the new value. Child rows of an audit event.

3.1 Why the audit trail is two tables rather than one

A single PATCH request that changes both the cleaning method and the notes is one act, by one person, at one instant, affecting two fields. Splitting the audit trail into a header and its lines models that directly: the header stores who and when once, and each changed field becomes a child row. The retrieval endpoint can then return grouped history — "Priya Nair updated 2 fields at 14:02" with the two field diffs nested beneath it — without any grouping logic in the query.

The flat alternative, one row per field change with the actor and timestamp repeated on every row, is marginally simpler to write. It was rejected because reconstructing an act from those rows requires grouping on a timestamp, which becomes ambiguous the moment two users save within the same instant. This trade-off is recorded in NOTES.md.

4. Entity-Relationship Diagram

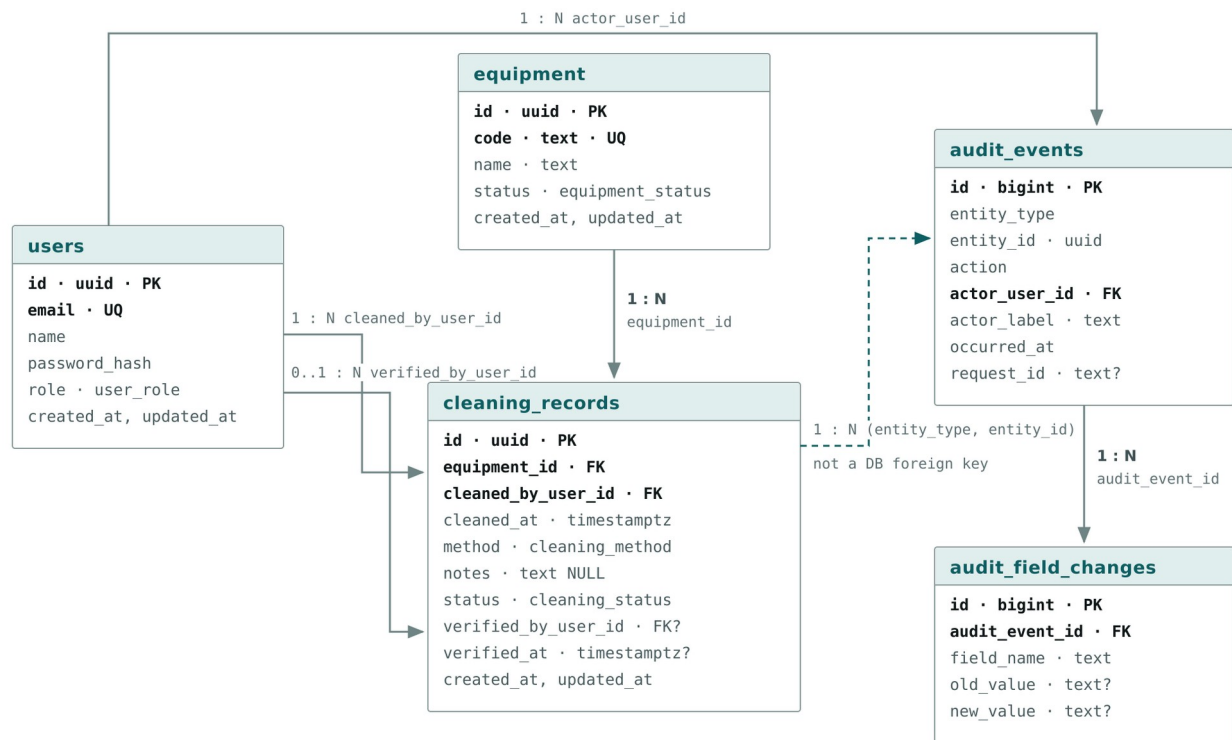


Figure 1 — Entity relationships. Solid arrows are enforced foreign keys, drawn from the "one" side to the "many" side. The dashed arrow is the single relationship the database does not enforce.

4.1 Relationships

Relationship	Cardinality	Notes
equipment → cleaning_records	1 : N	ON DELETE RESTRICT. Cleaning history must outlive any attempt to remove the asset.
users → cleaning_records (cleaner)	1 : N	cleaned_by_user_id. Defaults to the authenticated user, but may name a colleague who performed the work.
users → cleaning_records (verifier)	0..1 : N	verified_by_user_id. Null until the record is verified.
users → audit_events (actor)	1 : N	ON DELETE RESTRICT. An actor can never be removed from under their own history.
cleaning_records → audit_events	1 : N	Addressed by the pair (entity_type, entity_id) rather than a foreign key, so the same audit tables can later cover other entities without migrating existing history. Referential integrity for this pair is enforced in application code, which is why cleaning records are never hard-deleted.
audit_events → audit_field_changes	1 : N	ON DELETE CASCADE. A field-change line has no meaning without its header.

5. Enumerations

All five are native PostgreSQL enum types. They are code-owned closed sets; if cleaning methods later needed to be configurable per plant, the type would become a lookup table. That possibility is noted rather than pre-built.

Type	Values	Semantics and transitions
<code>equipment_status</code>	active, retired	Retired equipment is excluded from the cleaning-record creation picker but keeps its full history readable. Retirement replaces deletion.
<code>cleaning_status</code>	pending, verified	One-way transition: pending → verified. Un-verifying is not offered; a mistaken verification is corrected by a new record, which is the behaviour a regulated workflow expects.
<code>cleaning_method</code>	manual, cip, cop, solvent_flush	CIP is clean-in-place; COP is clean-out-of-place. Modelled as an enum rather than free text so that an audit diff of the method field compares members of a closed set.
<code>audit_action</code>	create, update	There is deliberately no delete member — nothing in this system is deleted.
<code>audit_entity</code>	cleaning_record	Extension point for the polymorphic audit target. Only <code>cleaning_record</code> is wired up today.
<code>user_role</code>	operator, supervisor, admin	Only supervisor and admin may move a cleaning record to verified.

6. Column-Level Schema

6.1 users

Column	Type	Constraints	Notes
<code>id</code>	<code>uuid</code>	PK, default <code>gen_random_uuid()</code>	Supplied by <code>pgcrypto</code> . Carries no ordering meaning.
<code>email</code>	<code>citext</code>	NOT NULL, UNIQUE	Case-insensitive login identity.
<code>name</code>	<code>text</code>	NOT NULL	Display name. Snapshotted into <code>audit_events.actor_label</code> at the time of each change.
<code>password_hash</code>	<code>text</code>	NOT NULL	<code>argon2id</code> . Never included in any list projection.
<code>role</code>	<code>user_role</code>	NOT NULL, default 'operator'	Governs who may verify a cleaning record.
<code>created_at</code>	<code>timestamptz</code>	NOT NULL, default <code>now()</code>	
<code>updated_at</code>	<code>timestamptz</code>	NOT NULL, default <code>now()</code>	Maintained in the service layer rather than by a trigger, so the behaviour is visible in the application code.

6.2 equipment

Column	Type	Constraints	Notes
<code>id</code>	<code>uuid</code>	PK	

Column	Type	Constraints	Notes
code	text	NOT NULL, UNIQUE, CHECK length 2–32	Asset tag, for example MIX-101. A unique violation is mapped to HTTP 409, not a 500.
name	text	NOT NULL, CHECK length 1–120	
status	equipment_status	NOT NULL, default 'active'	
created_at, updated_at	timestampz	NOT NULL	

Index: (status, code) supporting the paginated equipment list.

6.3 cleaning_records

Column	Type	Constraints	Notes
id	uuid	PK	
equipment_id	uuid	NOT NULL, FK → equipment(id) ON DELETE RESTRICT	RESTRICT rather than CASCADE: history must survive an attempted deletion of the asset.
cleaned_by_user_id	uuid	NOT NULL, FK → users(id) ON DELETE RESTRICT	The stretch goal's real cleanedBy. Defaults to the authenticated user.
cleaned_at	timestampz	NOT NULL	Client-supplied, since a cleaning is often logged after the fact. The not-in-the-future rule is enforced in the service layer, because now() is not immutable and cannot appear in a CHECK constraint.
method	cleaning_method	NOT NULL	
notes	text	NULL, CHECK length ≤ 2000	Nullable by design — it is the field that exercises null-to-value and value-to-null diffing.
status	cleaning_status	NOT NULL, default 'pending'	
verified_by_user_id	uuid	NULL, FK → users(id)	
verified_at	timestampz	NULL	
created_at, updated_at	timestampz	NOT NULL	There is no created_by column: the create audit event already records the author.

Table constraint: (status = 'verified') = (verified_by_user_id IS NOT NULL AND verified_at IS NOT NULL) — the verified state cannot exist half-populated.

Indexes: (equipment_id, cleaned_at DESC, id DESC) for keyset pagination, and (equipment_id, status, cleaned_at DESC, id DESC) for the status-filtered variant.

6.4 audit_events

Column	Type	Constraints	Notes
id	bigint	PK, GENERATED ALWAYS AS IDENTITY	Monotonic by intent. The table is append-only, high-volume, and read in strict chronological order, so a sequential integer supplies both the ordering guarantee and the pagination cursor. occurred_at alone cannot order events, since two events can share a timestamp.
entity_type	audit_entity	NOT NULL	Which kind of row was changed.
entity_id	uuid	NOT NULL	Which specific row. Together with entity_type this is the polymorphic pointer; no foreign key.
action	audit_action	NOT NULL	Distinguishes a creation, where every field moved from nothing, from an update.
actor_user_id	uuid	NOT NULL, FK → users(id) ON DELETE RESTRICT	The "who", as a referenced identity.
actor_label	text	NOT NULL	A frozen copy of the actor's name and email at the instant of the change. A join would return the actor's name today, not their name then; if a user is later renamed, every historical audit view would silently rewrite itself. The foreign key provides integrity, this column provides historical accuracy.
occurred_at	timestamptz	NOT NULL, default now()	Server clock only, never accepted from the client. Contrast cleaned_at, which is a claim about the physical world; occurred_at is a fact about the system.
request_id	text	NULL	Correlates an audit row with the request log line that produced it. Nullable because seed and migration writes have no HTTP request behind them.

Index: (entity_type, entity_id, id DESC). Immutability: the application database role is granted only INSERT and SELECT on both audit tables, so an accidental UPDATE fails at the database rather than quietly rewriting history.

6.5 audit_field_changes

Column	Type	Constraints	Notes
id	bigint	PK, IDENTITY	
audit_event_id	bigint	NOT NULL, FK → audit_events(id) ON DELETE CASCADE	
field_name	text	NOT NULL, UNIQUE (audit_event_id, field_name)	Validated against a code-side allowlist of auditable fields. One field cannot appear twice within one event.
old_value	text	NULL	An empty prior value is stored as SQL NULL, never as the string "null".

Column	Type	Constraints	Notes
new_value	text	NULL	Canonical text rendering: enum members verbatim, timestamps as ISO-8601 UTC, user references as the referenced label.

Trade-off: text keeps the table display-ready and trivially comparable, at the cost of losing the original type. A jsonb column would preserve types and require a rendering layer on read. Recorded in NOTES.md.

6.6 Worked example of stored audit data

Three acts against one cleaning record over a single day: Priya creates it at 09:14, corrects the method and adds a note at 11:02, and Ravi, a supervisor, verifies it at 14:30. That is three acts, so three audit events.

id	entity_type	action	actor_label	occurred_at
41	cleaning_record	create	Priya Nair <priya@...>	2026-09-08 09:14:22+05:30
42	cleaning_record	update	Priya Nair <priya@...>	2026-09-08 11:02:47+05:30
43	cleaning_record	update	Ravi Kumar <ravi@...>	2026-09-08 14:30:05+05:30

The corresponding field-change rows — one per field that actually moved:

id	audit_event_id	field_name / old_value	new_value
88	42	method · manual	cip
89	42	notes · NULL	Rinse extended to 12 min
90	43	status · pending	verified
91	43	verified_by_user_id · NULL	Ravi Kumar
92	43	verified_at · NULL	2026-09-08T09:00:05Z

Event 42 has two children because one save changed two fields; event 43 has three. Read together they render directly as the audit panel: "11:02 · Priya Nair · updated — method: manual → cip; notes: — → Rinse extended to 12 min".

7. The Audited Write Path

This is the mechanism the assignment identifies as the primary evaluation criterion, so the order of operations is stated explicitly. What follows is the inside of a single PATCH /api/cleaning-records/:id handler.

7.1 Why the sequence is shaped this way

Without an audit requirement the handler would be a single UPDATE statement. The audit requirement makes that impossible, because the UPDATE destroys the very information the audit trail needs: once it has run, the previous value of the changed field is gone and cannot be recovered. The audit trail therefore forces a fixed order — read the current row, compare it against the incoming patch, then write both the change and its history. Every remaining element of the design exists to protect that sequence.

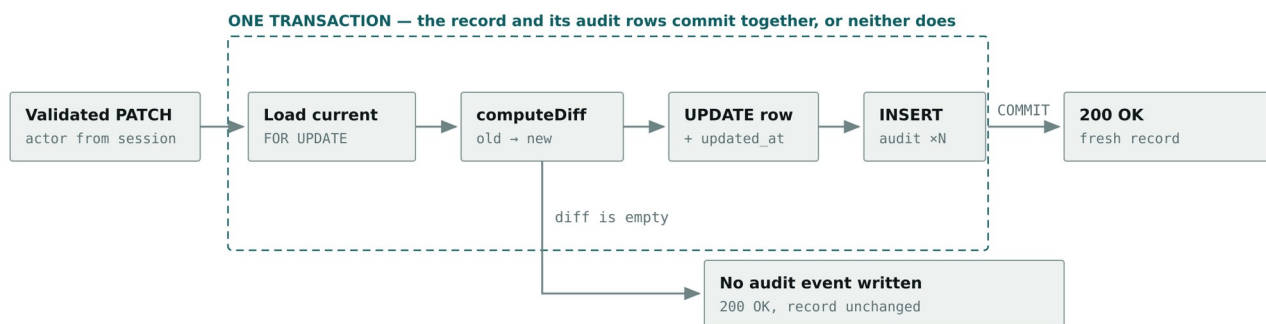


Figure 2 — The update path. The dashed boundary is a single database transaction; the lower branch is the no-op case.

7.2 Step by step

- **Request and authentication.** The body carries only the fields being changed. It does not carry the actor: the identity is read from the session token, because a client able to name its own actor could forge audit history.
- **Validation.** Zod parses the body, query, and path parameters before any database work, so a malformed request never opens a transaction or occupies a pooled connection.
- **BEGIN — open a transaction.** The update of the record and the insertion of its audit rows must either both happen or neither happen. Without a transaction, a failure between them would leave a record that had silently changed with no trace of who changed it — the worst available outcome in an auditable system, because unlike an outright failure it is invisible. The failure path becomes ROLLBACK: the update is undone, the caller receives a 500, and the row is exactly as it was.
- **SELECT ... FOR UPDATE — read and lock.** The SELECT supplies the old values needed for the comparison. FOR UPDATE locks that single row until the transaction ends, which prevents a specific corruption: two supervisors editing concurrently would otherwise both read the same original value and each write an audit line starting from it, producing a history that claims two independent changes from one starting point when the real sequence was serial. With the lock, the second transaction waits, then reads the already-updated row and records the correct intermediate value.
- **computeDiff — compare.** A pure function over the current row and the patch, with no database or framework dependency, which is what makes it inexpensive to test exhaustively.
- **The empty-diff branch.** A user who opens the edit form and saves without altering anything submits every field unchanged, and the diff is empty. The transaction commits, releasing the lock, and returns early. Writing an audit event with no field-change children would render as "Priya updated this record" with nothing beneath it, and months of those turn the audit panel into noise a reviewer has to read past.
- **Write, then COMMIT.** One UPDATE, one INSERT returning the new audit event id, and one set-based INSERT of the field-change rows. COMMIT makes all of it visible at once and releases the lock.

7.3 The diff function

```
const AUDITABLE = [
  'cleaned_by_user_id', 'cleaned_at', 'method',
  'notes', 'status', 'verified_by_user_id',
] as const

export function computeDiff(current: Row, patch: Patch): FieldChange[] {
  const changes: FieldChange[] = []

  for (const field of AUDITABLE) {
    if (!(field in patch)) continue          // not touched by this request

    const before = canonical(current[field])
    const after  = canonical(patch[field])

    if (before === after) continue          // submitted, but identical

    changes.push({ field, oldValue: before, newValue: after })
  }

  return changes
}
```

- **The "in" check carries PATCH semantics.** An absent key means leave the field alone; a key explicitly set to null means clear it. Setting notes to null must produce an audit line, while omitting notes must not. Testing key presence rather than an undefined value is what separates those two cases.
- **canonical() normalises before comparing.** A Date instance and its ISO-8601 string denote the same instant but are not equal by identity. Without normalisation every save would record phantom changes.
- **AUDITABLE is an allowlist, not a denylist.** The identifier, the equipment reference, and the timestamps are never audited. updated_at in particular changes on every save, so auditing it would append a meaningless line to every event.

7.4 The service function

```
export async function updateCleaningRecord(
  recordId: string,
  patch: CleaningRecordPatch,
  actor: AuthenticatedUser,
  requestId: string,
) {
  const client = await pool.connect()
  try {
    await client.query('BEGIN')

    const { rows } = await client.query(
      `SELECT * FROM cleaning_records WHERE id = $1 FOR UPDATE`,
      [recordId],
    )
    const current = rows[0]
    if (!current) throw new NotFoundError('cleaning_record', recordId)

    const changes = computeDiff(current, patch)

    if (changes.length === 0) {
      await client.query('COMMIT')          // release the lock, write nothing
      return current
    }
  }
```

```

const { rows: [updated] } = await client.query(
  `UPDATE cleaning_records
    SET method = COALESCE($2, method),
        notes = CASE WHEN $3::boolean THEN $4 ELSE notes END,
        updated_at = now()
    WHERE id = $1
    RETURNING *`,
  [recordId, patch.method ?? null, 'notes' in patch, patch.notes ?? null],
)

const { rows: [event] } = await client.query(
  `INSERT INTO audit_events
    (entity_type, entity_id, action, actor_user_id, actor_label, request_id)
    VALUES ('cleaning_record', $1, 'update', $2, $3, $4)
    RETURNING id`,
  [recordId, actor.id, `${actor.name} <${actor.email}>`, requestId],
)

await client.query(
  `INSERT INTO audit_field_changes
    (audit_event_id, field_name, old_value, new_value)
    SELECT $1, * FROM UNNEST($2::text[], $3::text[], $4::text[])`,
  [
    event.id,
    changes.map(c => c.field),
    changes.map(c => c.oldValue),
    changes.map(c => c.newValue),
  ],
)

await client.query('COMMIT')
return updated
} catch (err) {
  await client.query('ROLLBACK')
  throw err
} finally {
  client.release()
}
}

```

One detail specific to using the pg driver directly: every statement runs on the same acquired client, not on `pool.query()`. `BEGIN` applies to a connection, so if a later statement took a different pooled connection it would execute outside the transaction and every guarantee above would be lost. This is a real hazard of choosing raw SQL over an ORM, and is handled by acquiring the client once and passing it through the repository calls.

Field group	Audit behaviour
cleaned_by_user_id, cleaned_at, method, notes, status, verified_by_user_id	Audited. A change to any of these produces a field-change row.
id, equipment_id, created_at, updated_at	Never audited. Immutable, structural, or mechanical.

8. API Surface

Endpoint	Behaviour
POST /api/auth/login	Exchanges credentials for a signed httpOnly session cookie. Rate-limited.
GET /api/auth/me	Returns the current user, or 401.
POST /api/equipment	Creates equipment. 409 on duplicate code.
GET /api/equipment	Paginated list, optional status filter.
GET /api/equipment/:id	Single equipment, 404 if absent.
PATCH /api/equipment/:id	Partial update.
DELETE /api/equipment/:id	Retires the asset by setting status to retired. 409 if already retired. The row is never removed.
POST /api/equipment/:id/cleaning-records	Creates a cleaning record for that asset. 404 on unknown equipment, 409 on retired equipment. Writes a create audit event.
GET /api/equipment/:id/cleaning-records	Paginated list for that asset, newest cleaning first, optional status filter.
PATCH /api/cleaning-records/:id	Updates a record and writes the audit trail in the same transaction.
GET /api/cleaning-records/:id/audit	Audit history for one record, newest event first, each event carrying its field-change lines. Paginated.
GET /api/healthz	Process and database reachability.

8.1 Pagination contract

Every list endpoint uses keyset pagination rather than offset. The client sends an optional opaque cursor and a limit; the server returns the page together with the cursor for the next one.

```
GET /api/equipment/:id/cleaning-records?status=pending&limit=20&cursor=<opaque>
```

```
{
  "data": [ /* ... */ ],
  "nextCursor": "eyJjbGVhbmVkaXQ0IiYMDI2LTA5LTA4VDA5OjE0OjIyWiIsImlkIjo...",
  "hasMore": true
}
```

- **The cursor encodes (cleaned_at, id)**, base64-encoded and opaque to the client. The query predicate compares that tuple against the same composite index that satisfies the ORDER BY, so each page costs $O(\text{limit})$ regardless of how deep it is — an offset query must scan and discard every preceding row.
- **A unique tiebreaker is always present.** Ordering by cleaned_at alone is not deterministic when two records share a timestamp, and rows could then shuffle between pages. Ordering by (cleaned_at DESC, id DESC) removes that entirely.
- **Correctness under concurrent writes.** A row inserted between two page fetches neither duplicates an already-seen row nor hides an unseen one, because the cursor anchors to a value rather than to a position. This is the property offset pagination cannot provide, and it is covered by a dedicated test.
- **limit defaults to 20 and is clamped to 100.** A malformed or tampered cursor returns 400 rather than silently resetting to the first page.

9. Functional Requirements

Each requirement is traced to its origin: Core is required by the assignment, Stretch is one of the assignment's optional goals, and Derived is a decision taken where the assignment left the choice open.

9.1 Data model and database

ID	Requirement	Source
DM-1	Equipment table with id, code, name, status (active/retired) and timestamps; code unique.	Core
DM-2	CleaningRecord table belonging to equipment, with cleanedBy, cleanedAt, method, notes and status (pending/verified).	Core
DM-3	Audit event header storing actor, server timestamp, action and target entity.	Core
DM-4	Audit field-change rows storing field name with old value and new value.	Core
DM-5	User table so that cleanedBy and the audit actor are foreign keys rather than strings.	Stretch
DM-6	Verifier columns with a CHECK constraint binding them to the verified status.	Derived
DM-7	Forward-only SQL migration files, committed to the repository, runnable with one command.	Core
DM-8	Seed script: several users, six equipment items, and enough cleaning records on one asset (at least 60) to page through, plus pre-existing audit history to demonstrate.	Core
DM-9	Composite indexes supporting every paginated query, including the status-filtered variant.	Derived

9.2 API

ID	Requirement	Source
API-1	Create equipment.	Core
API-2	List equipment, paginated, with an optional status filter.	Core
API-3	Read a single equipment item; 404 when absent.	Core
API-4	Partial update of equipment.	Core
API-5	Delete endpoint retires the asset rather than removing the row; 409 if already retired.	Derived
API-6	Create a cleaning record for a given equipment; 404 on unknown equipment, 409 on retired equipment.	Core
API-7	List cleaning records for a given equipment, paginated, newest cleaning first.	Core
API-8	Update a cleaning record, writing the audit trail.	Core
API-9	Fetch audit history for a record, newest event first, each event carrying its field-change lines.	Core
API-10	Keyset pagination: opaque cursor over (cleaned_at, id), limit defaulting to 20 and capped at 100, response returning data, nextCursor and hasMore.	Stretch
API-11	Status filter on the cleaning-record list, composable with pagination.	Core
API-12	Invalid or tampered cursors return 400, never a silent reset to the first page.	Derived
API-13	Audit rows are written in the same transaction as the mutation they describe.	Core
API-14	A mutation whose diff is empty writes no audit event.	Derived
API-15	Diffing covers only the auditable-field allowlist, compares canonical string forms, and is null-safe in both directions.	Core

ID	Requirement	Source
API-16	The record is locked FOR UPDATE before diffing, so concurrent updates cannot interleave into incorrect history.	Derived
API-17	Login and current-user endpoints; a signed httpOnly cookie identifies the session.	Stretch
API-18	All mutating routes require authentication (401 otherwise); the audit actor comes from the token, never from the request body.	Stretch
API-19	Only supervisor and admin roles may move a record to verified (403 otherwise) — segregation of duties.	Derived
API-20	Every body, query and path parameter is parsed by a Zod schema; failures return 400 with a per-field details array.	Stretch
API-21	A single error envelope across the API, with a documented mapping for 400, 401, 403, 404, 409, 422 and 500.	Stretch
API-22	Domain rule checks: cleanedAt not in the future, notes length, and legality of the status transition.	Stretch
API-23	Health endpoint reporting process and database reachability.	Derived

9.3 Front-end

ID	Requirement	Source
FE-1	Equipment list showing code, name and a status badge; selecting an item opens its detail view.	Core
FE-2	Equipment detail showing that asset's cleaning records, paginated via the keyset cursor.	Core
FE-3	Status filter on the record list that resets pagination when changed.	Core
FE-4	Form to add a cleaning record (cleanedBy, cleanedAt, method, notes).	Core
FE-5	The same form in edit mode, pre-filled from the existing record.	Core
FE-6	Audit trail panel: one block per event showing actor, timestamp and action, containing a row per changed field rendered old → new, with empty values shown as an explicit dash.	Core
FE-7	Verify action on a pending record, visible only to users whose role permits it.	Derived
FE-8	Login screen and a current-user indicator with sign-out.	Stretch
FE-9	Server field errors mapped back onto the form fields that caused them.	Stretch
FE-10	Explicit loading, empty and error states for every list and panel; a failed mutation leaves the user's input in place.	Derived
FE-11	Typed API client sharing the server's request and response types, with no duplicated hand-written interfaces.	Derived

9.4 Delivery

ID	Requirement	Source
OPS-1	Docker Compose brings up PostgreSQL, the API and the web client; migrations and seed run on startup.	Stretch
OPS-2	A non-Docker path is also documented: exact commands for install, database creation, migrate, seed, and running each side.	Core
OPS-3	Environment templates for both applications; no secrets committed.	Core
OPS-4	README covering what the system is, how to run it, the API surface, the seeded logins, and how to run the tests.	Core

ID	Requirement	Source
OPS-5	NOTES.md recording decisions and trade-offs, the keyset-versus-offset rationale, assumptions made, and what was deliberately left out.	Core
OPS-6	Commit history in readable steps — schema, API, audit, front-end, tests, documentation.	Core

10. Non-Functional Requirements

ID	Quality	How it is met
NFR-1	Type safety	TypeScript strict mode on both sides, with noUncheckedIndexedAccess. No use of any in domain code. Row shapes are declared once and mapped explicitly at the repository boundary rather than cast.
NFR-2	Injection safety	Every query is parameterized. No user-supplied value is concatenated into SQL, including sort and filter inputs, which are resolved through allowlists.
NFR-3	Audit immutability	The application database role holds only INSERT and SELECT on the audit tables, so an accidental UPDATE fails at the database rather than quietly rewriting history.
NFR-4	Attributability	The actor is stored as both a foreign key and a frozen text label; audit timestamps come from the server clock; actors cannot be deleted; records are retired rather than removed.
NFR-5	Time handling	timestampz throughout, stored in UTC, ISO-8601 at the API boundary, rendered in the viewer's local zone with the zone displayed.
NFR-6	Query performance	List reads cost $O(\text{limit})$: the keyset predicate runs against an index matching the ORDER BY exactly, so there is no deep-page scan. Verified with EXPLAIN against the seeded data.
NFR-7	Deterministic ordering	Every paginated query orders by a non-unique column plus a unique tiebreaker, so records sharing a timestamp can never shuffle between pages.
NFR-8	Error hygiene	One error shape with machine-readable codes. No stack traces, SQL text or constraint names appear in responses; unexpected errors are logged with a request id and returned as a generic 500.
NFR-9	Security baseline	argon2id password hashing, httpOnly SameSite cookie with a short expiry, helmet, CORS restricted to the web origin, login rate limiting, and a request body size cap.
NFR-10	Observability	Structured request logs carrying a request id that is also written to the audit event, so any audit row can be traced back to the call that produced it.
NFR-11	Test realism	API tests run against a real PostgreSQL instance with each test wrapped in a transaction that is rolled back at teardown. The database is never mocked for pagination or audit tests, since those are exactly the behaviours a mock would fabricate.
NFR-12	Layering	route → schema → service → repository. No SQL in handlers and no HTTP types in services. computeDiff is a pure function with no framework or driver imports, which is what makes its test suite inexpensive.
NFR-13	Developer experience	One command to run everything, plus separate migrate, seed and test scripts and a single verify script that runs typecheck, lint and tests together.
NFR-14	Accessibility	Labelled inputs, errors associated with their fields, the audit trail as a real table with header cells, visible keyboard focus, and full keyboard operation.
NFR-15	Concurrency	Row lock before diff; updated_at returned to the client and accepted back as a precondition, so a stale form cannot silently overwrite a newer edit.
NFR-16	Migration discipline	Numbered, forward-only, safe to re-run. The schema is reproducible from an empty database in one command with no manual step.

ID	Quality	How it is met
NFR-17	Bounded responses	No endpoint can return an unbounded set; the audit endpoint is paginated on its identity column under the same cursor contract as the record list.
NFR-18	Restraint	The assignment asks for no over-building. No component library, no state-management library beyond a query cache, and no abstraction introduced for a single caller. Scope is frozen at the requirements above.

11. Test Coverage

The suite is deliberately small and aimed at the two areas the assignment names: audit-diff correctness and pagination correctness.

ID	Area	What is asserted
TST-1	computeDiff, unit	No change; one field; several fields; null to value; value to null; a field submitted with an identical value; a non-auditable field ignored; and timestamp representations that are equal but differently formatted.
TST-2	Audit correctness	A create produces exactly one create event; an update produces one event whose field-change lines match the fields that moved and nothing else.
TST-3	Pagination completeness	Walking every page of a seeded set yields each row exactly once, in order, with no gaps.
TST-4	Keyset under concurrency	A row inserted between two page fetches neither duplicates nor hides an existing row — the property offset pagination fails.
TST-5	Pagination edges	limit clamped to the maximum; malformed cursor returns 400; filter combined with cursor; an empty result returns hasMore false and no cursor.
TST-6	Atomicity	A forced failure while inserting audit rows leaves the record itself unchanged.
TST-7	Authorisation	A mutation without a session returns 401; the recorded actor equals the session user even when the body claims otherwise; an operator attempting to verify receives 403.
TST-8	Validation	A malformed payload returns 400 naming the offending field, and writes nothing.

12. Assumptions

The assignment invites reasonable assumptions where it is ambiguous. These are the ones taken; each is also recorded in NOTES.md.

- **A verified record remains editable**, with the change audited. A fully regulated implementation would lock it behind an electronic signature; that is out of scope, and the audit trail is what makes a later edit acceptable here.
- **Deleting equipment means retiring it**. The assignment asks for CRUD, but a hard delete would either orphan cleaning history or cascade it away, and both are wrong for an auditable system.
- **Cleaning records are never deleted**, which is why the audit action enum has no delete member.
- **cleanedBy is a user reference rather than free text**. The assignment's example suggests a string; taking the authentication stretch goal makes it a foreign key, while still allowing an operator to log a cleaning performed by a colleague.

- **cleanedAt is client-supplied** and may be in the past, since cleanings are often entered after a shift, but may not be in the future.
- **Audit values are stored as canonical text**, with the rendering rule for each type written down so that the diff is deterministic.
- **Audit history is returned newest-first and paginated**, even though an individual record will rarely accumulate many events.
- **Seed users have known passwords**, printed in the README, so that a reviewer can sign in within a minute of starting the application.

13. Exclusions

The following were considered and deliberately left out, to keep the submission within the suggested effort and the assignment's instruction not to over-build.

- Electronic signatures and signature manifestation under 21 CFR Part 11. The audit trail is the slice requested; a compliance product is not.
- Equipment-level audit history. The schema's `entity_type` column supports it; only cleaning records are wired up.
- Cleaning-validation scheduling, due-date alerting, swab or TOC sample results, file attachments, and batch or campaign linkage.
- A full role-based permission matrix. One role check — who may verify — is enforced.
- Password reset, self sign-up, refresh-token rotation, and session management.
- Reporting, CSV export, and printable batch records.
- A CI pipeline, monitoring, and any deployment target beyond Docker Compose.
- Optimistic UI updates. Mutations refetch instead; at this scale correctness is worth more than perceived speed.

14. Deliverables

Artefact	Contents
Git repository	Full source for the API and web client, migrations, seed script, tests, and the Compose definition. Commit history structured by area rather than squashed.
README.md	Exact steps to install, create and migrate the database, seed it, and run both sides — with and without Docker. Includes the API surface and the seeded credentials.
NOTES.md	Key decisions and trade-offs, the keyset-versus-offset rationale, the assumptions in section 12, the exclusions in section 13, and what would come next with more time.
This document	The design record: entity model, ER diagram, schema detail, the audit mechanism, and the traced requirement set.

Prepared by Aravind R · aravind.r194@gmail.com · 08 September 2026